CLIFFT RESEARCH · JULY 2026

Two ways past the wall

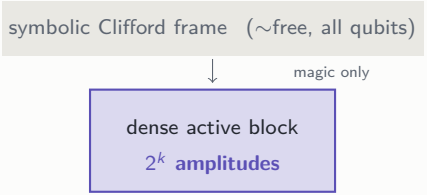
Extending clifft beyond its 2^k limit

A · Stabilizer-rank backend
change the *representation*:
pay for magic, not for dimensions

B · GPU shot batching
change the *hardware use*:
run thousands of shots abreast

clifft in thirty seconds

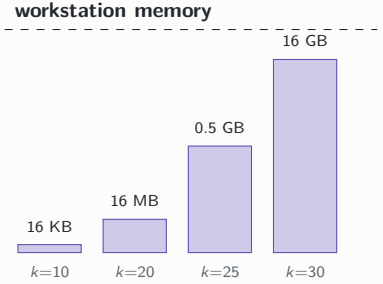
- Clifford gates are "easy": clifft tracks them **symbolically**, almost for free, however many qubits.
- Only genuine *magic* (T gates) forces it to hold real amplitudes: a dense array of 2^k complex numbers, the **active block**.
- k counts how much magic is live at once — usually far smaller than the qubit count. That is why clifft is fast.



One engine, two tiers: symbolic where physics allows it, dense only where it must be.

The wall: 2^k doubles with every step

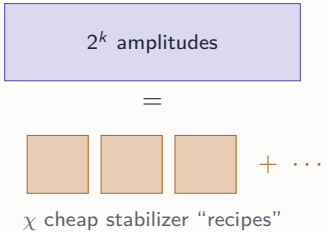
- Memory doubles per unit of k: $k = 30 \Rightarrow 16$ **GB**, $k = 40 \Rightarrow 16$ TB.
- clifft's compiler already fights hard for small k (measurement-driven reduction) — circuits that still land at large k are *genuinely* dense.
- So the remaining question is not "compile better" but: **what do we do when 2^k itself is the cost?**



Two independent bets on what to do about it — one mathematical, one architectural.

A · The idea: pay for magic, not for dimensions

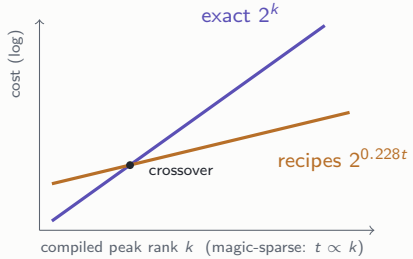
- States with *little* magic are secretly simple: they are a **short sum of "free" stabilizer states** — the states Clifford simulators handle natively.
- Store χ small recipes (a few KB each) instead of 2^k raw numbers. Like describing an image as a few overlapping patterns instead of listing every pixel.
- Clifford gates update all recipes cheaply; only T gates grow their number; **measurements shrink it back**.



The cost driver becomes the amount of magic t, not the dimension 2^k.

A · Why it should work

- There is an optimal way to split a T gate so the number of recipes grows only as $2^{0.228\,t}$ — far slower than 2^k when magic is sparse.
- Accepting a small, **tunable** error δ lets you keep only a random sample of recipes ($\sim 1/\delta^2$ of them). Measured accuracy is *predictable*: error $= 0.50\,\delta$, on the nose.
- Everything stays in the language clifft already speaks: same frame idea, same measurement collapse.



Trade exactness you can measure for reach you cannot otherwise have.

A · Evidence (measured, against clifft's real fast path)

- Every component validated to **machine precision** against exact references (incl. an independent checker we built).
- Head-to-head on circuits that compile to *genuinely large* blocks (peak rank $\approx n/2$): the backend overtakes exact clifft at $n \approx 34$ (coarse) and $n \approx 48$ (8% error).
- Past $n \approx 62$ it is the **only method that runs at all**.
- External exact tools on the same circuits: QuiZX needs 25× longer by n=56; Quokka# (model counting) **times out beyond** n=24.

$n = 72$	memory	time
any exact method	∼1 TB	—
rank backend	**1.9 GB**	**267 s**

Accuracy dial, measured:
error $= 0.50\,\delta$ across the whole sweep
(halve the error ⇒ 4× the work).

The niche is real — and honestly bounded: below the crossover, exact clifft stays the right tool.

A · Where it wins, where it doesn't

Small peak rank	Large peak rank, sparse magic	Adaptive, Clifford-heavy
Not qubit count: clifft compiles most circuits, however wide, to a *small* block — then it is faster *and* exact. Keep it.	**The niche.** Even the *compiled* block breaks the memory wall, T-count low: the only feasible method (approximate, dial-controlled).	**Real protocols win**: cultivation-style patches 4–16×, teleported-T gadgets $\sim$2×; bare syndrome extraction sits at the boundary. The standard open-source tool cannot run these circuits at all.

- Also measured where it *loses*: with no logical Clifford traffic the frame's bookkeeping costs up to $\sim$2× — the advantage is a measured phase boundary, not a slogan.

Status: prototype validated; honest benchmarks, natural workloads, external baselines done.

A · It generalizes: any Clifford+T circuit

- The trick for circuits with magic *everywhere*: teleport each in-line T onto its own helper qubit (the same adaptive gadget real hardware uses) — then all magic sits in **one up-front layer** again.
- So the sampler keeps its best properties on *any* circuit: **no penalty for interleaving**, and the normalization stays exactly known — the expensive estimation step never enters.
- Tested at scale on circuits whose answer is known *by construction* (hidden shift: the output is the hidden string, always): at up to **96 total qubits**, the backend returns 0.90–0.99 of the true probability 1, in seconds.
- Honest finding: clifft's compiler crushes many such circuits to tiny cores and answers in microseconds — which turns "who wins" into a *rule*, not a race → next slide.

Generality at the product-magic price — validated end to end against exact references.

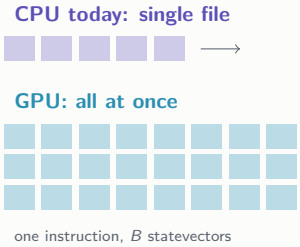
A · One compiler, two executors

- Both cost exponents are known from the compiled program **before anything runs** — dispatch is a one-line comparison, per program.
- **Wired, not just proposed**: the backend runs clifft's *optimized* circuits — compilation absorbs *every* Clifford and halves the live T-count (validated to 10^{-13}).
- The backend is an explicit opt-in (δ), never a silent substitute: clifft's exactness contract stays intact.

clifft compiles; the cheaper engine runs — on the same optimized circuit.

B · The idea: don't run one shot faster — run 4,096 abreast

- A clifft gate just *streams* a small array (16 KB–16 MB). One such array cannot keep a GPU busy: **a stadium choir asked to sing one sentence**.
- But clifft's workloads are Monte-Carlo: **thousands of shots of the same program**, today run one after another.
- Put B=4,096 shots' arrays on the GPU side by side and apply every gate to *all of them* in one launch.



Batching doesn't make the GPU faster — it makes the problem big enough for the GPU to be fast.

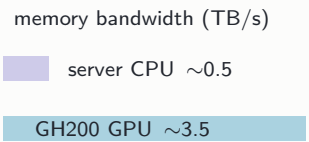
B · Why it should work (1): shots march in lockstep

- The shape of a shot — which operation runs when, and how big the array is at every step — is **fixed at compile time and identical for every shot**. clifft's compiler already writes this schedule down.
- What differs per shot (measurement outcomes, noise) is just **a few numbers**, which enter the shared computation as per-shot parameters — like SIMD lanes, one lane per shot.
- So the classic objection to batching — "shots diverge" — *does not apply to clifft*. Its structure is unusually batchable.
- And nobody serves this niche: the two existing batched-GPU simulators both **fall back to one-shot-at-a-time** exactly when circuits measure mid-flight — clifft's home turf.

The hardest architectural question resolves in the design's favor before any GPU code is written.

B · Why it should work (2): the speed limit is bandwidth — and GPUs have 7× more

- These operations do almost no arithmetic per byte: on *any* machine they run at the speed of memory.
- Measured: clifft on 11 CPU cores is only 2.6× one core — **the bus is the limit, not the cores**.
- A GH200's memory is ∼7× wider than a server CPU's. Once the batch saturates it: **5–10× more shots per second**, honestly bounded.



big published "100×" numbers use weak baselines — we keep a refuted-claims list and quote the physics.

The claim is deliberately modest — which is why we can trust it.

B · The one risk, and the $20 experiment that settles it

- Mid-circuit measurements need a decision from the CPU: results cross to the host, outcomes cross back. Each crossing costs microseconds — **the same scale as the small kernels themselves**.
- Two reasons it should survive: the crossing is paid **once per batch** (nanoseconds per shot at B=4,096), and the GH200's CPU–GPU link is built precisely to make it cheap.
- A microbenchmark faithful to clifft's kernels — *including complete measurements with the round-trip* — is built, validated, and waiting. GH200 time: $2/hour, self-serve.

round-trip measured cheap	**go**: batched backend; first target = replay-style queries (no round-trips at all)
round-trip eats the win	**no-go**, cleanly — or on-device sampling is the follow-up

Either answer is cheap, quantitative, and final — that is what the microbenchmark is for.

How the two bets fit together

A · Stabilizer rank extends *reach*

circuits clifft cannot hold at all today — approximate, with a measured accuracy dial

B · GPU batching extends *throughput*

the circuits clifft already runs — many more shots per second, exact as ever

- Both work *because of* clifft's structure, not despite it: the symbolic frame, the small active block, and the compile-time shot schedule are exactly what decompositions and batching need.
- Both were reviewed adversarially; every headline number above survived a fair baseline.

Next: A — benchmarks complete; write it up. B — the GH200 run, then batched replay queries.